A Secure Local Storage Protection Scheme for Flutter Applications Against Reverse Engineering

Prepared by

Mohammed Abdullah Mohammed Musaed Nasser

Supervisor

Assoc. Prof. Dr. Ahmed Al-Shalabi

13 May 2026

Research Proposal for the Master's Degree in Cyber Security

ABSTRACT

Flutter applications often retain tokens, cached records, preference values, and SQLite data after a session ends. Release builds compile Dart code ahead of time, but that compilation does not encrypt the application's files. The storage remains a separate attack surface. An analyst with access to the package, its data directory, or the running process can examine embedded constants, copy records, or intercept data during decryption.

This study will design and evaluate a local storage scheme for Flutter applications. The implementation will combine envelope encryption, authenticated payload encryption, scoped key derivation, and record-context authentication. Android Keystore will protect the root wrapping key; the iOS design will specify Keychain access conditions separately. Integrity checks and instrumentation signals may supplement these mechanisms, with their effects measured independently.

I will implement the scheme as a reusable Flutter plugin and compare it with five reference configurations. The experiments will cover package inspection, Dart AOT analysis, storage extraction, runtime hooks, record modification, rollback, and repackaging. Measurements will include recovered plaintext, reusable keys, accepted modifications, read and write latency, memory use, package size, and integration effort. A verification matrix will link the observed behavior to applicable OWASP MASVS controls and MASTG procedures. The report will identify the points where encrypted data becomes accessible during legitimate application execution.

Keywords

Flutter security; secure local storage; reverse engineering; envelope encryption; Android Keystore; iOS Keychain; AEAD; HKDF; mobile application security; OWASP MASVS; OWASP MASTG.

Contents

Abstract **1**
Keywords 1

Proposal At-a-Glance **5**

Alignment with the Supervisor's Chapter Guides **6**

Chapter 1: Introduction and Research Foundation **7**
1.1 Background and Context 7
1.2 Research Problem 7
1.3 Scientific Gap 7
1.4 Purpose Statement / Aim 8
1.5 Research Questions 8
1.6 Research Objectives 9
1.7 Research Hypothesis 9
1.8 Significance of the Study 9
1.9 Scope and Delimitations of the Study 10
1.10 Limitations of the Study 10
1.11 Definition of Key Terms 11
1.12 Organization of the Thesis 11

Chapter 2: Literature Review and Theoretical Background **13**
2.1 Introduction 13
2.2 Literature Search Logic and Screening Criteria 13
2.3 Flutter Architecture and Local Storage Models 14
2.4 Flutter Reverse Engineering and Runtime Analysis 14
2.5 Secure Local Storage and Platform-Backed Key Management 15
2.6 Database Encryption and SQLCipher Baselines 15
2.7 Envelope Encryption, AEAD, and Key-Wrapping Foundations 16
2.8 Runtime Instrumentation and Attack-Based Evaluation 16
2.9 Standards and Official Technical References 16
2.10 Comparative Review of Previous Studies and Technical References 16
2.11 Refined Synthesis and Identified Research Gap 18
2.12 Conceptual Framework of the Proposed Study 18

Chapter 3: Research Methodology and Experimental Design **19**
3.1 Research Design 19
3.2 Quantitative Nature of the Study 19
3.3 Proposed Artifact and System Architecture 19
3.4 Technical Design of the Proposed Scheme 19
3.5 Threat Model and Assumptions 20

3.6 Unit of Analysis and Observation 21
3.7 Baseline Approaches and Planned Comparison 22
3.8 Variables and Metrics 22
3.9 Research Question, Objective, and Metric Mapping 23
3.10 Experimental Environment 23
3.11 Attack Scenarios 24
3.12 Experimental Protocol 25
3.13 Data Collection and Evidence Logging 25
3.14 Data Analysis Plan 26
3.15 Statistical Treatment and Repetition Plan 27
3.16 Validity, Reliability, and Reproducibility 27
3.17 Ethical Considerations 28
3.18 Expected Results 28
3.19 Proposed Timeline 28

Proposal Readiness Checklist **29**

Conclusion **29**

List of Figures

1 Research gap and positioning of the proposed study. 8
2 Proposed secure local storage architecture for Flutter applications. 20
3 Threat model and attack surfaces in the proposed Flutter local storage workflow. 21
4 Proposed research methodology and attack-based evaluation pipeline. 25
5 Evaluation scenario and comparative baseline framework. 26

PROPOSAL AT-A-GLANCE

The research follows one storage lifecycle: data enters the application, is encrypted and written, then is retrieved and decrypted. Each stage has a different exposure. The proposal defines a plugin, five comparison configurations, attack procedures, and measurements for that lifecycle.

Element	Research decision
Core problem	Sensitive SQLite records, files, preferences, caches, and tokens can remain readable or depend on keys exposed elsewhere in the application.
Research gap	The study will evaluate the interaction of Flutter execution, platform key management, encrypted payloads, and record-context checks under a shared set of attack conditions.
Proposed artifact	A Flutter plugin with documented storage APIs, an envelope format, key-lifecycle rules, and a reference application.
Evaluation	Run supported workloads across the proposed scheme and five baselines, recording the same access conditions and attack criteria.
Main measurements	Plaintext and reusable-key recovery, tamper acceptance, latency, CPU and memory use, package size, integration effort, and control-verification evidence.
Boundaries	Android local storage is the primary test target. Backend security, network protocols, malware detection, and kernel exploitation remain outside the experiments.

ALIGNMENT WITH THE SUPERVISOR'S CHAPTER GUIDES

Guide requirement	Location	Treatment in this proposal
Problem statement	Chapter 1, Sections 1.2–1.3	Defines exposed storage paths, key-handling weaknesses, and the evaluation question.
Purpose statement	Chapter 1, Section 1.4	Identifies the artifact to build and the outcomes to compare.
Research questions	Chapter 1, Section 1.5	Connects storage weaknesses, attack outcomes, overhead, integration, and standards verification.
Objectives	Chapter 1, Section 1.6	Specifies the research tasks needed to answer the questions.
Significance	Chapter 1, Section 1.8	Describes the expected research, implementation, and testing outputs.
Scope and limitations	Chapter 1, Sections 1.9–1.10	States the supported platforms, attacker capabilities, and limits of the findings.
Literature review	Chapter 2	Groups sources by the part of the storage and attack workflow they address.
Methodology	Chapter 3	Specifies configurations, workloads, measurements, evidence, analysis, validity, and research ethics.

CHAPTER 1: INTRODUCTION AND RESEARCH FOUNDATION

1.1 Background and Context

Mobile applications retain authentication tokens, profile records, offline business objects, and preferences to restore state or continue work without a connection. Flutter applications can write this information to SQLite, files, shared preferences, or caches. The same value may pass through several of these locations before it is removed.

Flutter's Android release builds compile Dart application code into native instructions. This changes the tools an analyst needs to inspect the application. It does not change the stored data format. Obfuscation can obscure Dart names, while a database row or cached token remains readable if the application wrote it without encryption.

Local protection therefore depends on how data is serialized, which key encrypts it, where that key is held, and what the application does after decryption. A token returned to Dart code may remain observable even when its stored representation is encrypted.

1.2 Research Problem

A Flutter application may protect a token through a secure-storage plugin and still write the same token to a log or cache. Database encryption can have a similar weakness: the file is encrypted, but its passphrase is embedded in the application or retrieved through an interceptable execution path. Studies of hardcoded credentials and secrets distributed in mobile applications provide related evidence for examining package contents alongside stored records [25, 26].

The proposed evaluation will examine plaintext persistence, exposed key material, nonce handling, and the association between an encrypted value and its storage location. The attacker may inspect resources, copy data files, alter records, repackage the application, or hook a decryption operation. The prerequisites for each action will be recorded.

Existing mechanisms cover different operations. Flutter obfuscation changes code identifiers. Secure-storage plugins expose protected key-value APIs. SQLCipher encrypts a database, while application code supplies its key. Integrity checks can detect some modifications, although their result can itself become a hook target.

The problem is how these mechanisms interact in a Flutter storage implementation. I will trace the paths that carry keys and plaintext between the platform layer, the plugin, and application code, then test the stored artifacts and those execution paths separately.

1.3 Scientific Gap

The related work covers Flutter reversing, encrypted SQLite, platform key storage, and runtime extraction. Each provides part of the technical basis for this study. A combined implementation still needs evidence about record substitution, key retrieval, migration, and plaintext exposure during use.

This proposal addresses that evaluation problem through a Flutter plugin with a specified key hierarchy and envelope format. The literature review will compare the proposed functions with existing packages and studies. Any claim of contribution will be tied to those comparisons and the experiments performed on the implementation.

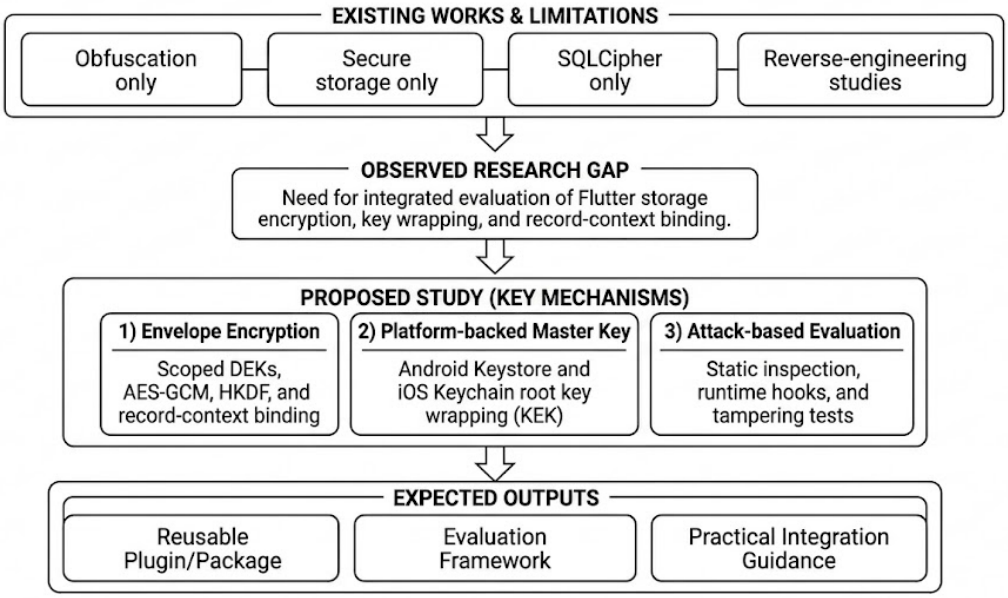


Figure 1: Research gap and positioning of the proposed study.

1.4 Purpose Statement / Aim

The purpose of this quantitative design-science study is to build and test a local storage scheme for Flutter applications under defined reverse-engineering conditions. I will compare the implementation with five reference configurations, measuring data and key recovery, modification handling, performance, and integration effort. The final verification matrix will relate the evidence to applicable MASVS controls and MASTG tests.

1.5 Research Questions

- **RQ1.** Which local storage and secret-handling weaknesses can the selected analysis workflows identify in the Flutter test application?
- **RQ2.** How do obfuscation, secure key-value storage, SQLCipher, and field encryption behave under matched workloads and attack conditions?
- **RQ3.** How does envelope encryption with platform-managed wrapping keys and authenticated record context affect stored-data recovery, reusable-key exposure, and modification acceptance?
- **RQ4.** What performance costs and integration requirements arise in the selected Flutter storage workflows?

- **RQ5.** Which applicable OWASP MASVS requirements does the implementation satisfy, and what MASTG test evidence supports each finding?

1.6 Research Objectives

1. Define the storage assets, attacker capabilities, and access assumptions for the Flutter test application.
2. Review relevant studies, platform documentation, and tools for AOT inspection, encrypted storage, key management, and runtime extraction.
3. Specify a key hierarchy and envelope format using authenticated encryption, scoped derivation, record-context checks, and platform key protection.
4. Implement the design as a Flutter plugin with documented defaults and integration examples.
5. Compare security outcomes, resource use, and integration effort across the proposed scheme and five baselines.
6. Produce a control-verification matrix that links the implementation and experimental evidence to the selected MASVS and MASTG requirements.

1.7 Research Hypothesis

The proposed scheme is expected to reduce plaintext recovery from extracted storage relative to plaintext and obfuscation-only configurations. Platform-managed wrapping keys should reduce reusable root-key recovery relative to implementations that embed or persist those keys in accessible application artifacts. Record-context authentication should reject ciphertext moved to a different identifier, table, or user scope.

Runtime interception may produce a different result. The tests will measure whether an attacker can recover an unwrapped DEK or capture a payload after legitimate decryption. Performance and integration results will be reported alongside these security outcomes.

1.8 Significance of the Study

Dimension	Expected contribution
Theoretical significance	Describes how Flutter execution paths, key boundaries, and persistent data representations interact under the defined attacker model.
Practical significance	Produces a plugin and integration examples for the storage operations implemented in the study.
Methodological significance	Supplies comparable artifact-extraction, binary-inspection, hooking, and modification procedures with recorded evidence.
Security significance	Measures plaintext and key recovery and checks whether modified or substituted records are accepted.

Dimension	Expected contribution
Developer significance	Documents initialization, migration, error handling, and key rotation, including configuration choices that affect protection.

1.9 Scope and Delimitations of the Study

The study concerns sensitive data retained by Flutter mobile applications in databases, files, preferences, caches, and token stores. Android will provide the primary experimental environment. The iOS component will describe Keychain integration and may include an implementation if suitable devices and tooling are available; any iOS tests will be reported separately.

Included in the study	Excluded from the study
Flutter mobile persistence operations and their storage artifacts.	Backend compromise and cloud infrastructure security.
SQLite content, application files, preference values, caches, and token-like records.	Network protocol design and transport-security testing.
Encryption at rest, key management, record-context authentication, and modification handling.	General malware detection and operating-system hardening.
Static analysis, AOT inspection, artifact extraction, runtime hooks, storage modification, and repackaging.	Kernel exploitation and malicious operating-system components.
Android measurements and an architectural treatment of iOS, with optional separately reported iOS tests.	Claims covering every commercial application, device, operating-system release, or attacker.

1.10 Limitations of the Study

Tool support depends on the Flutter engine, Dart snapshot format, Android release, and processor architecture. A script that locates a function in one build may need adjustment for another. The report will record those dependencies and unsuccessful tool operations.

Hardware key protection also varies by device. Android Keystore access alone will not be treated as evidence that the key resides in a TEE or StrongBox. The measured configuration will include the key's reported security level.

The test application will use controlled workflows and synthetic content. Results will apply to those workflows and attack procedures. A successful hook after decryption can recover usable data without exporting the root key, and the study will record that outcome as plaintext exposure during execution.

1.11 Definition of Key Terms

Term	Operational definition in this study
Secure local storage	Encryption, key management, and authentication checks applied to sensitive application data written to persistent storage.
Reverse engineering	Static or dynamic inspection of packaged code, resources, stored artifacts, and application behavior.
Envelope encryption	Payload encryption using DEKs, with those keys wrapped under a separate KEK.
Authenticated encryption with associated data (AEAD)	Encryption that verifies the ciphertext and specified unencrypted context before the application accepts the payload.
Platform-backed key protection	Operating-system facilities that protect keys or secrets and enforce their access conditions; exportability and hardware backing depend on the platform and configuration.
Baseline approach	A documented reference configuration used to compare storage behavior and costs.
Tampering resistance	Detection or rejection of modified ciphertext or mismatched record context. Acceptance of an older valid record is assessed separately as rollback behavior.
Runtime instrumentation	Hooks, tracing, or memory inspection applied to the running application using tools such as Frida and Objection.

1.12 Organization of the Thesis

Chapter	Planned content
Chapter 1	Problem, contribution to investigate, purpose, questions, objectives, scope, limitations, and terminology.
Chapter 2	Search approach, Flutter architecture, reversing tools, encrypted storage, key management, cryptographic references, and related-work comparisons.

Chapter	Planned content
Chapter 3	Artifact, baselines, attacker model, workloads, measurements, attack procedures, evidence, analysis, validity, and ethics.
Chapter 4	Plugin implementation, key hierarchy, envelope format, platform interfaces, migration, and reference application.
Chapter 5	Experimental outcomes, performance measurements, baseline comparisons, and discussion of the observed limits.
Chapter 6	Findings, deployment recommendations, study limitations, and remaining research questions.

CHAPTER 2: LITERATURE REVIEW AND THEORETICAL BACKGROUND

2.1 Introduction

The review follows the components involved in a storage operation: Flutter code, platform interfaces, payload encryption, key access, and the return of decrypted values. It draws on Flutter-specific work where available and on Android and cryptographic sources for the mechanisms used by the plugin.

Sources will be compared by their implementation assumptions and evaluation conditions. A paper about database encryption may inform key handling without testing Dart buffers; a reversing tool may expose application structure without evaluating the protection of stored records.

2.2 Literature Search Logic and Screening Criteria

The search is organized by technical component. Queries for Flutter reversing, SQLite encryption, platform keys, and runtime interception cover different parts of the proposed implementation. This approach also includes related Android studies whose results concern the key or database interfaces used by Flutter plugins.

Search block	Role in the study	Representative terms
Flutter reverse engineering	Identify the analysis methods and build-dependent limits relevant to release applications.	Flutter reverse engineering, Dart AOT, snapshots, `libapp.so`, B(l)utter, reFlutter, Frida.
Flutter secure storage	Examine package interfaces and platform-specific storage options.	Flutter secure storage, `flutter_secure_storage`, Android Keystore, iOS Keychain, shared preferences.
SQLCipher and encrypted SQLite	Specify a database-encryption baseline and its key-retrieval path.	SQLCipher, encrypted SQLite, Android databases, passphrase handling, key management.
Keystore, wrapping, and envelope encryption	Support the separation of payload keys from root wrapping keys.	Android Keystore, StrongBox, key wrapping, DEK, KEK, HKDF, envelope encryption.
Runtime instrumentation	Locate methods for observing keys and plaintext during application operations.	Frida, Objection, hooks, dynamic analysis, forensic decryption, memory inspection.
Standards and testing guidance	Select applicable controls and procedures for verification.	OWASP MASVS, MASTG, MASVS-STORAGE, MASVS-CRYPTO, MASVS-RESILIENCE.

This is a structured literature review at the proposal stage. It does not use search-result counts to establish novelty. The record will identify the query focus, source type, and reason each source was retained.

Database or source type	Search focus	Screening role
Google Scholar and scholarly search	Discovery across Flutter security, encrypted storage, key management, and runtime analysis.	Locate candidate sources and trace related references.
IEEE Xplore and ACM Digital Library	Peer-reviewed research on mobile security, database protection, instrumentation, and obfuscation.	Prioritize studies that provide relevant implementations or measurable evaluations.
arXiv and technical preprints	Recent work on secure key storage and mobile security.	Retain relevant work with its publication status stated explicitly.
OWASP and official documentation	Platform API behavior, storage controls, SQLCipher, AES-GCM, and HKDF.	Establish configuration constraints and verification requirements.

Flutter-specific sources will be screened for their relevance to storage, keys, binary analysis, or instrumentation. Android studies will be retained when they address a mechanism used by the proposed plugin. UI design, cloud-storage architecture, and performance reports without a local-security component will be excluded from the core comparison.

2.3 Flutter Architecture and Local Storage Models

A Flutter application combines Dart code, engine libraries, an Android or iOS host, assets, and plugins. Platform channels can carry requests and values between Dart and native code. Persistence may use SQLite wrappers, object stores, files, preferences, caches, or secure-storage packages.

On Android, release application instructions are packaged in `libapp.so`. Other resources and plugin code remain available through their respective package formats. Local data is written later to application storage and must be examined separately from the distributed APK.

Flutter documents that obfuscation renames symbols without encrypting resources [1]. The storage evaluation will therefore vary obfuscation independently of encryption and retain the build flags used for each application variant.

2.4 Flutter Reverse Engineering and Runtime Analysis

Dart AOT snapshots and runtime conventions affect how Flutter code is analyzed. Batteux describes the internals relevant to reversing Flutter applications [7]. PNF Software provides a Dart AOT snapshot helper [8], while Wangwarunyoo's B(l)utter uses Dart runtime information

to assist analysis [9]. The experimental workflow will record the structures recovered by each tool and the engine versions it supports.

Wijaya and Kurniawan examine dynamic security analysis across Flutter versions [10]. Their work informs the use of application-layer observations alongside package inspection. This proposal includes log inspection and internal-storage extraction to check whether designated test values appear outside their intended protected representation.

Reports by Llewellyn, Rambhia and colleagues, and Thao and colleagues discuss vulnerabilities and protection techniques in Flutter applications [11, 12, 13]. The review will attribute findings to the tested applications and configurations, recording the storage mechanism, build mode, and extraction procedure where these are reported.

2.5 Secure Local Storage and Platform-Backed Key Management

Android Keystore can perform operations with non-exportable keys, and supported configurations can bind key material to a TEE or StrongBox [15]. The plugin will inspect the generated key's security level. A software-backed configuration will be identified in the results.

iOS Keychain protects secret items through access groups, accessibility classes, and access-control policies [16]. A retrieved symmetric secret may enter application memory. The iOS design will specify that boundary rather than assume that every Keychain item behaves like a non-exportable hardware key.

The `flutter_secure_storage` package exposes protected key-value operations [14]. It can supply a token or protected key material to application code, but its API does not provide database queries or relational transactions. File and database protection require additional storage handling.

KeyDroid examines secure key storage in Android applications [17]. Its analysis will inform the review of key initialization, access restrictions, hardware assumptions, and application-side handling. These checks will be applied to the wrapping key and to any unwrapped key material used by the plugin.

2.6 Database Encryption and SQLCipher Baselines

SQLCipher provides encrypted SQLite storage [20]. The application still supplies the database key. Key generation, persistence, retrieval, replacement, and use during queries determine the exposure beyond the database file itself.

Park and colleagues describe a secure database architecture for Android [18]. Akowuah and colleagues examine TrustZone protection for Android SQLite data [19]. Their designs will be reviewed for the placement of keys, the supported database operations, and the treatment of plaintext during use.

The comparison will record complete configurations. A SQLCipher variant with an embedded passphrase tests that passphrase choice; its outcome cannot establish the security of every SQLCipher integration. The experiments will separately report file confidentiality, key recovery, runtime interception, and modification handling.

2.7 Envelope Encryption, AEAD, and Key-Wrapping Foundations

Envelope encryption uses separate keys for payload encryption and key wrapping. A DEK protects the records in a defined storage domain. A KEK protects that DEK, and the resulting wrapped material can be persisted with the envelope parameters.

AES-GCM authenticates ciphertext and associated data [22]. Its use requires a nonce that is unique under the active key. The proposed context includes the namespace, table or file identifier, record identifier, schema version, user scope, and key version where applicable.

The reader must obtain the expected context from the requested storage operation. Accepting an attacker-supplied replacement context alongside a moved ciphertext would defeat the intended record binding. An older valid envelope at the same location can still pass authentication unless a separate freshness mechanism rejects it.

HKDF-SHA-256 will derive context-specific keys from defined input key material [23]. The implementation specification will document the salt, `info` encoding, and purpose of each derived key. The non-exportable KEK will remain an input to platform wrapping operations rather than be retrieved into Dart for derivation.

2.8 Runtime Instrumentation and Attack-Based Evaluation

Encrypted storage can be studied by observing the application while it decrypts data. Anglano and colleagues use a Frida-based framework for forensic examination of application-level encrypted data on Android [21]. Their approach informs hooks around key retrieval, cryptographic calls, and returned buffers.

Plaintext still enters the application process. If a hook captures a query result after successful decryption, the test will report that exposure even when the root key was never exported. DEK recovery, root-key recovery, and plaintext interception will have separate success criteria.

MASTG provides storage, cryptographic, and reverse-engineering procedures, including Flutter guidance [2, 3]. The selected procedures will be documented with the scripts, extracted artifacts, and observations produced during each run.

2.9 Standards and Official Technical References

MASVS-STORAGE addresses sensitive stored data and unintended leakage. MASVS-CRYPTO covers cryptographic use and key management, and MASVS-RESILIENCE covers resistance to reverse engineering and tampering [4, 5, 6]. The evaluation will identify which controls apply to the plugin and which depend on the host application's behavior.

Platform documentation will constrain the key-protection interfaces [15, 16]. SQLCipher documentation will define the database baseline [20]. NIST SP 800-38D and RFC 5869 will provide the technical basis for GCM and HKDF [22, 23]. The report will retain the selected versions and configuration options.

2.10 Comparative Review of Previous Studies and Technical References

The comparison separates analysis tools, storage architectures, and verification references. Each source is assessed for the part of the implementation or test procedure it informs.

A. Flutter reverse-engineering and Flutter security sources

Study or source	Relevant focus	Relationship to this research
Batteux, 2022 [7]	Dart AOT internals and Flutter reversing.	Informs inspection of compiled instructions and runtime structures.
PNF Software, 2022/2023 [8]	Tool support for Dart AOT snapshot analysis.	Provides a reference for snapshot-assisted binary inspection and tool-version limits.
Wangwarunyoo, 2023 [9]	Flutter analysis using Dart runtime information.	Informs recovery of available symbols and structural information.
Wijaya and Kurniawan, 2025 [10]	Dynamic security analysis across Flutter versions.	Informs application-layer observations and version-dependent evaluation.
Flutter security studies [12, 13]	Flutter and cross-platform security techniques.	Supply background for the comparison; their implementations and test coverage will be examined individually.

B. Local storage, database encryption, and key-management sources

Study or source	Relevant focus	Relationship to this research
Park et al., 2018 [18]	Secure database architecture on Android.	Informs the database baseline and its key-management assumptions.
Akowuah et al., 2018 [19]	TrustZone protection for Android SQLite data.	Informs the treatment of hardware boundaries and database operations.
SQLCipher documentation [20]	Encrypted SQLite implementation and configuration.	Defines the database-encryption baseline; application key handling remains part of the test configuration.
Flutter secure storage [14]	Platform-specific protected key-value APIs.	Defines a supported key-value baseline and relevant package options.
KeyDroid, 2025 [17]	Secure key storage in Android applications.	Informs checks for key creation, access policy, hardware backing, and application-side exposure.

C. Runtime instrumentation, standards, and evaluation sources

Study or source	Relevant focus	Relationship to this research
Anglano et al., 2023 [21]	Frida-based examination of application-encrypted Android data.	Informs interception tests around key access and decryption.
OWASP MASTG and MASVS [2, 4, 5, 6]	Mobile security controls and test procedures.	Provide the selected verification criteria and evidence mapping.

Study or source	Relevant focus	Relationship to this research
NIST SP 800-38D [22]	GCM and GMAC.	Constrains authenticated-encryption use, including nonce and tag handling.
RFC 5869 [23]	HKDF extract-and-expand derivation.	Defines the derivation inputs and context-separation mechanism.
Wang et al., 2018 [24]	Empirical study of mobile obfuscation.	Provides context for the obfuscation baseline and its distinction from persistent-data encryption.

2.11 Refined Synthesis and Identified Research Gap

The reviewed sources address different boundaries. Flutter analysis tools examine the released binary. Secure-storage packages expose protected values through application APIs. SQLCipher protects the database representation. Platform key stores constrain root-key access, while runtime instrumentation can observe operations carried out with those keys.

The study will evaluate those boundaries within one Flutter implementation. The comparison will test whether record-context checks reject substituted data, whether a recovered artifact can be opened with an extracted key, and where plaintext becomes available during normal use. Existing work and packages will be assessed against the same functions before the thesis makes a novelty claim.

2.12 Conceptual Framework of the Proposed Study

The framework links storage representation, key handling, and runtime exposure to measured outcomes. Five baselines will isolate common implementation choices: plaintext storage, obfuscation, protected key-value storage, SQLCipher, and field encryption.

The proposed scheme adds coordinated wrapping, scoped keys, and authenticated record context. Extraction and modification tests will measure its behavior, while benchmarks and integration records will quantify the cost of those choices. The verification matrix will identify the applicable controls supported by the evidence.

CHAPTER 3: RESEARCH METHODOLOGY AND EXPERIMENTAL DESIGN

3.1 Research Design

The project uses a quantitative design-science method. I will first specify and implement the plugin, then evaluate it through controlled attack procedures and storage workloads. The artifact and five baselines will use the same reference application where their interfaces support the required operation.

The evidence will include recovery outcomes, modification acceptance, timings, resource measurements, and integration records. Screenshots and tool logs will document how each numerical outcome was obtained.

3.2 Quantitative Nature of the Study

Methodological element	Decision for this study
Study family	Evaluation of a software artifact and storage-security mechanisms.
Main evidence	Attack results, operation timings, resource readings, extracted files, tool logs, and control-verification records.
Comparison	Proposed scheme and five documented baselines tested under matched supported operations.
Measurement rule	Define each outcome and its success criterion before collecting results.
Reproducibility	Retain SDK and dependency versions, build settings, device specifications, workload definitions, and scripts.

3.3 Proposed Artifact and System Architecture

The artifact will be a Flutter plugin that handles selected sensitive database records, files, and key-value data. Its public API will separate storage operations from platform key access. The native layer will perform operations requiring the protected root key.

The implementation will document where DEKs are unwrapped and where payloads cross the native and Dart interfaces. Any copies created during serialization, channel transfer, or returned-value handling will be considered in the instrumentation tests.

3.4 Technical Design of the Proposed Scheme

- **Root-key protection:** Generate the Android KEK through Keystore and record its security level and permitted uses. Specify Keychain accessibility and access policy separately for the iOS design.

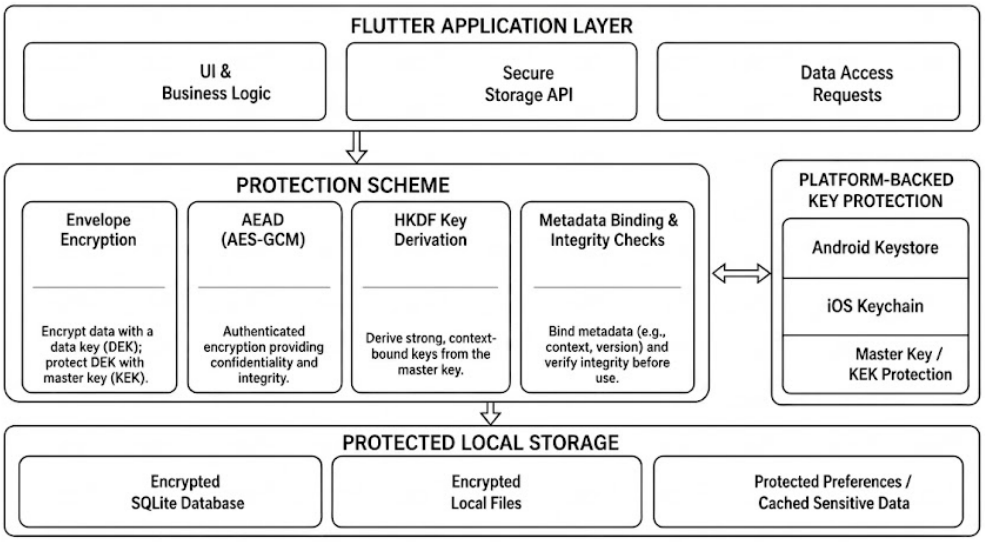


Figure 2: Proposed secure local storage architecture for Flutter applications.

- **Envelope encryption:** Use domain DEKs for payload encryption and wrap them under the KEK. Persist the wrapped material with the parameters needed to identify the envelope and active key version.
- **Authenticated encryption:** Use AES-256-GCM as the primary primitive. ChaCha20-Poly1305 remains an alternative if platform constraints require it; any change will be specified and evaluated as a separate configuration. The handler will withhold the payload when authentication fails.
- **Key derivation:** Use HKDF-SHA-256 with specified input key material, salt, and `info` fields to separate domain and key-purpose contexts.
- **Metadata binding:** Authenticate the expected table or file identifier, record ID, schema version, key version, and user scope where applicable. Test substitution separately from replay of an older valid record.
- **Rotation:** Version the KEKs, DEKs, and envelope format. Test gradual migration, access to existing records, interrupted updates, and handling of missing or invalidated keys.

3.5 Threat Model and Assumptions

Component	Definition in this study
Assets	Sensitive SQLite records, files, caches, tokens, operational secrets, record context, wrapped keys, and relevant configuration values.

Component	Definition in this study
Attackers	Analysts who possess the package and can run the controlled application on an emulator or rooted Android device. Any iOS attacker conditions will be specified separately.
Capabilities	Package inspection, AOT analysis, permitted storage extraction, application-process hooks, record modification, and repackaging.
Security objectives	Reduce recoverable stored plaintext and reusable key material; detect changed ciphertext or mismatched record context; document runtime exposure.
Exclusions	Kernel exploitation, malicious backends, phishing, and guarantees of secure deletion beyond platform behavior.

The access prerequisites matter for each test. Ordinary ADB access or physical possession will not be assumed to reveal protected application directories. Extraction runs will record the root, debugging, backup, or other access condition actually used.

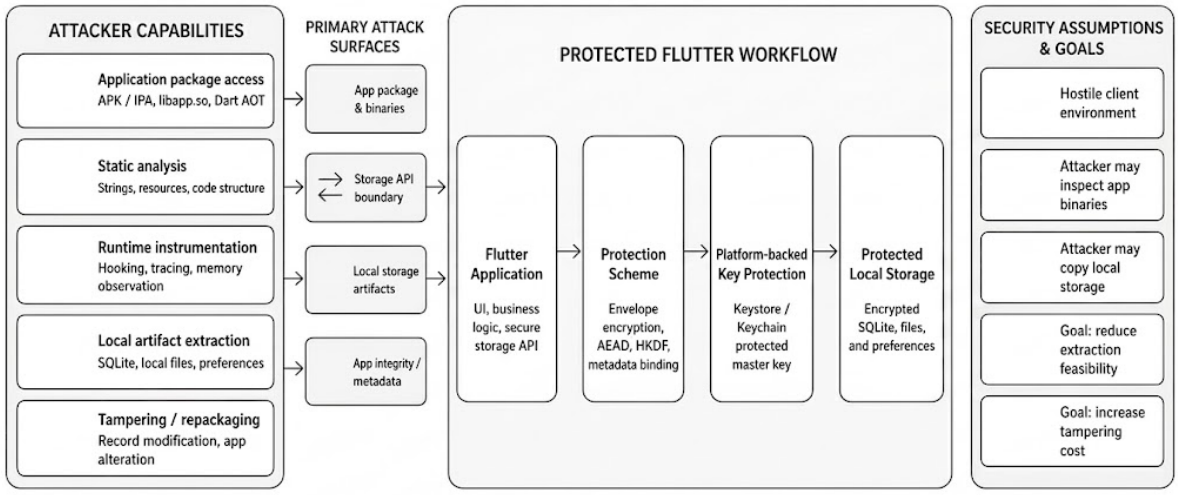


Figure 3: Threat model and attack surfaces in the proposed Flutter local storage workflow.

3.6 Unit of Analysis and Observation

The unit of analysis is a storage operation or attack attempt executed under a recorded configuration. Each run will identify the application variant, storage type, payload size, scenario, device or emulator, and output.

The observations include recovered content, authentication decisions, timings, logs, screenshots, and extracted files. A recovered key will count as reusable only if it decrypts the corresponding extracted artifact under the documented test conditions.

3.7 Baseline Approaches and Planned Comparison

Approach	Protection scope	Exposure to test	Key handling	Evaluation role
No protection	Application sandbox and platform defaults.	Readable designated values after permitted extraction.	No application-level encryption key.	Plaintext reference.
Obfuscation only	Renamed Dart identifiers.	Plaintext storage and recoverable packaged values.	Same storage configuration as the plaintext variant.	Isolates build obfuscation.
Secure storage only	Protected key-value items.	Returned values, runtime key access, and package-specific settings.	Selected secure-storage version and platform options.	Key-value reference.
SQLCipher / full-DB encryption	Encrypted SQLite database.	Passphrase recovery and plaintext query results.	Passphrase source and retrieval path recorded explicitly.	Database-encryption reference.
Field-level encryption	Selected sensitive record fields.	Unprotected fields, application keys, and record substitution.	Documented application-managed key and metadata policy.	Selective-encryption reference.
Proposed scheme	Wrapped domain keys, authenticated payloads, and record-context checks.	Unwrapped DEKs, plaintext during use, accessible key operations, and migration failures.	Platform-managed KEK and versioned wrapped DEKs.	Research artifact.

I will compare matched operations where the APIs support them. A key-value package's lack of relational queries will be recorded as an interface limitation rather than an encryption failure. Each baseline will retain its own configuration record so that an exposed key can be traced to the retrieval or persistence choice that produced it.

3.8 Variables and Metrics

Variable type	Variables or measurements
Independent variables	Storage approach, attack scenario, storage type, payload size, and explicitly varied build or device conditions.

Variable type	Variables or measurements
Dependent variables	Plaintext and reusable-key recovery, modification acceptance, latency, startup cost, CPU and RAM use, package-size increase, and integration effort.
Controlled conditions	SDK, application workflows, dataset contents, build mode, environment, attack steps, and measurement tools held constant within each comparison.
Security measurements	Recovery success and failure, recovered material, authentication rejection, accepted modified records, and verified control coverage.
Performance measurements	Read and write latency, database open time, startup overhead, CPU utilization, memory use, and package-size change.
Integration measurements	Setup steps, configuration points, exposed API choices, migration work, and clarity of reported errors.

Where build or device conditions are varied, results will be grouped by that condition. They will be held constant for comparisons of storage approaches within the group.

3.9 Research Question, Objective, and Metric Mapping

Question	Aligned objective	Evidence or measurement
RQ1	Identify storage and secret-handling weaknesses in the test application.	Recovered artifacts, execution-path observations, and a categorized weakness record.
RQ2	Compare common protection configurations.	Recovery outcomes, accepted modifications, and the baseline matrix.
RQ3	Assess envelope encryption and platform key management.	Stored-plaintext recovery, reusable-key recovery, context-check rejection, and runtime interceptions.
RQ4	Measure implementation costs.	Operation latency, startup time, CPU and memory use, package size, and integration steps.
RQ5	Verify applicable standards requirements.	MASVS control matrix linked to MASTG procedures and retained test evidence.

3.10 Experimental Environment

Environment element	Planned specification
Application	A Flutter reference application containing synthetic tokens, cached records, SQLite data, files, and preference values.
Variants	Plaintext, obfuscation only, secure key-value storage, SQLCipher, field encryption, and the proposed scheme.
Platform	Android as the measured test platform; iOS considered architecturally with optional separately documented implementation and tests.
Tools	`apktool`, Jadx, `strings`, MobSF where applicable, Ghidra, Dart AOT analysis tools, Frida, Objection, and storage-extraction utilities.
Workloads	Small, medium, and larger payloads representing tokens, records, cached objects, and files; exact byte sizes recorded before execution.
Configuration record	Flutter and OS versions, dependencies, build flags, tool versions, device or emulator details, scripts, and dataset identifiers.

3.11 Attack Scenarios

The scenarios follow the exposure paths considered in the literature. The application-layer analysis described by Wijaya and Kurniawan informs the inclusion of log inspection and internal-storage checks [10]. The designated test values will be searched for in package artifacts, persisted files, logs, and intercepted buffers.

Scenario	Purpose	Evidence to retain
Static analysis	Inspect resources, strings, configuration, and host or plugin code.	Recovered constants or secrets, search results, and the inspected package version.
Binary inspection	Examine AOT instructions and available runtime structures.	Recovered identifiers, strings, relevant call locations, and unsupported analysis operations.
Local artifact extraction	Copy database files, preferences, caches, and envelopes under recorded access conditions.	Extracted files, designated plaintext matches, attempted decryption, and visible metadata.
Runtime instrumentation	Hook key access, unwrapping, encryption, decryption, and returned-value paths.	Hook scripts, intercepted material, call locations, and success or failure against the recovery criteria.
Tampering and replay	Alter ciphertext or context, swap records, and restore older artifacts.	Authentication failures, accepted changes, application errors, and separately recorded rollback acceptance.
Repackaging	Modify the controlled application package and attempt execution.	Build changes, integrity-check outcomes, hook bypasses, and accessible storage operations.

3.12 Experimental Protocol

Each run begins from a recorded state. The protocol will apply the same supported operation and attack criterion to each configuration, preserving implementation-specific parameters such as key retrieval and envelope layout.

1. Prepare synthetic tokens, records, files, and preferences; assign recognizable values and sensitivity categories.
2. Build the variants with the selected SDK and documented release flags.
3. Record the device or emulator state, dependency versions, tool versions, and key-security configuration.
4. Execute the predefined storage workload or attack procedure.
5. Retain outputs, extracted artifacts, screenshots, and measurements with a run identifier.
6. Repeat performance operations according to the warm-up and measurement plan.
7. Compare matched outcomes across the baselines and proposed scheme.
8. Link findings to the selected verification controls and identify the tested limits.

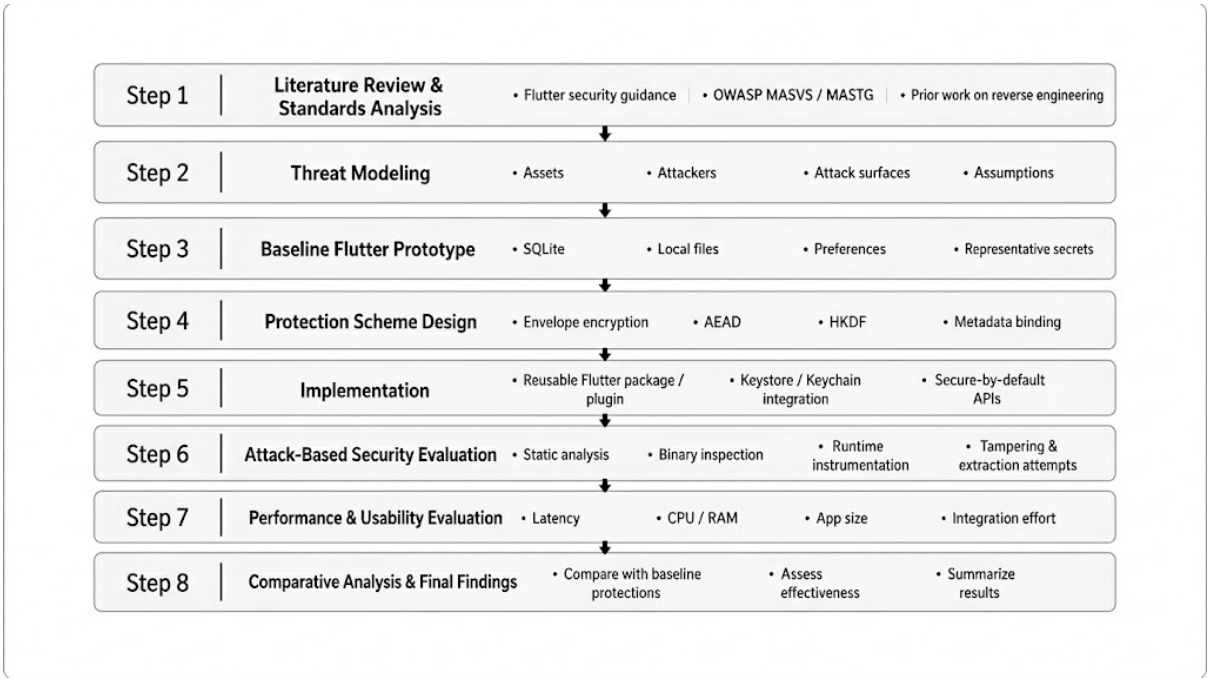


Figure 4: Proposed research methodology and attack-based evaluation pipeline.

3.13 Data Collection and Evidence Logging

The datasets will contain synthetic content. No real user account or production credential is required. Each token or record will have a known value so that a successful recovery can be checked against the input.

Evidence will include extracted files, hook outputs, logs, screenshots, timings, package sizes, and CPU and RAM readings. The index will identify the variant, scenario, run number, tool, configuration, and timestamp. Authentication failures and unexpectedly accepted records will be preserved with the altered artifact that produced them.

3.14 Data Analysis Plan

Analysis target	Planned analysis
Security outcomes	Compare designated plaintext recovery, reusable-key recovery, accepted modifications, rollback behavior, and authentication rejection.
Performance overhead	Summarize operation latency, database open time, startup cost, CPU and memory use, and package-size changes.
Integration effort	Record setup steps, configuration points, migration work, and key or metadata choices left to the application developer.
Standards verification	Link each applicable control to the implementation condition, test procedure, observed result, and supporting artifact.
Comparative interpretation	Explain which outcomes follow from encrypted storage, key handling, context checks, or interception after decryption.

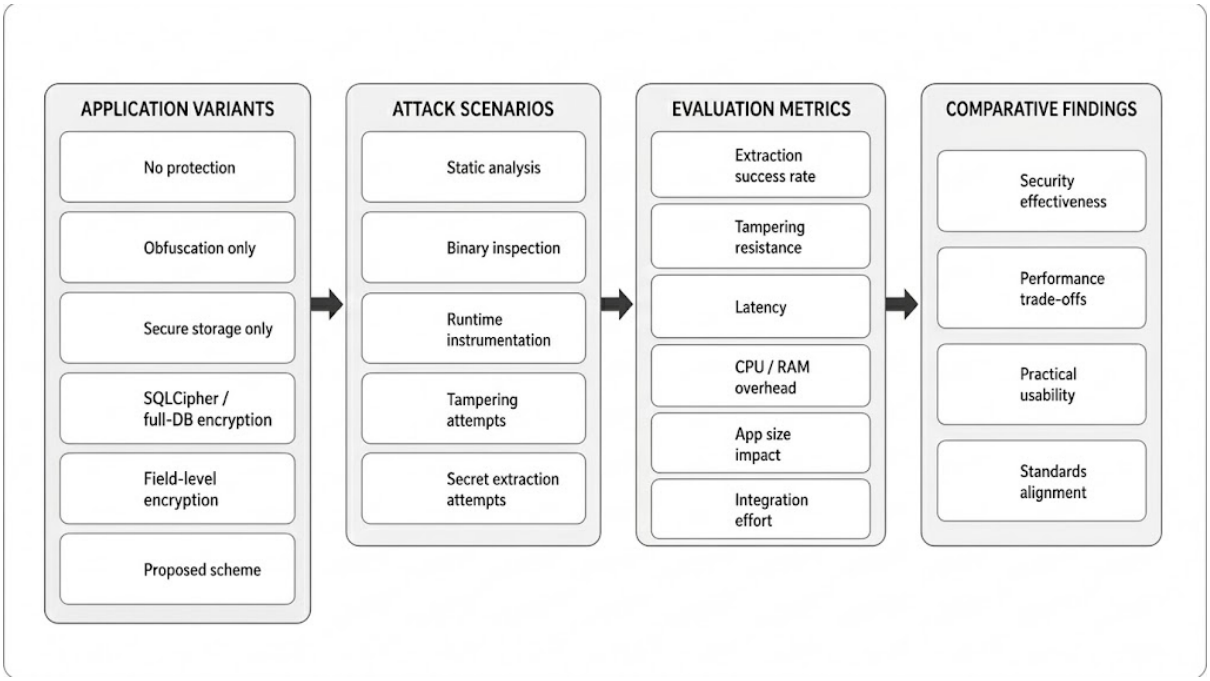


Figure 5: Evaluation scenario and comparative baseline framework.

3.15 Statistical Treatment and Repetition Plan

Attack outcomes will use predefined success criteria and stated denominators. A readable package string will not count as recovery of a protected record unless it matches the designated content. Key recovery will require successful use of the recovered material on the corresponding artifact.

Performance runs will be repeated for each supported operation, variant, and payload size. The initial target is five warm-up runs followed by 30 measured runs. Any departure from that target will be stated with the environment limitation and actual sample count.

Measurement item	Planned treatment
Warm-up and measured runs	Perform five warm-up runs and target 30 measured runs per supported operation, configuration, and data size. Record the achieved count.
Latency and resources	Report mean, median, standard deviation, minimum, maximum, and, where appropriate, a 95% confidence interval for the estimated mean.
Skewed or unstable readings	Report the median and interquartile range alongside the raw spread. Identify outliers and any exclusion rule applied.
Security outcomes	Report success and failure counts, percentages, the number of attempts, and the access conditions for each scenario.
Baseline comparisons	Use matched workflows and byte sizes, holding build, device, and tool conditions constant within a comparison.
Interpretation	Present recovery and modification outcomes beside measured overhead. State the workload and acceptance criterion used for any deployment recommendation.

Repeated execution of the same attack will be distinguished from a new configuration or hook attempt. The report will not treat repeated deterministic outcomes as evidence about an unrestricted population of attackers.

3.16 Validity, Reliability, and Reproducibility

Concern	Treatment
Internal validity	Hold workload and environment conditions constant within each comparison and document differences required by the storage interfaces.
Construct validity	Define plaintext recovery, reusable-key recovery, modification acceptance, and authentication rejection before testing.
External validity	Use representative storage operations and common analysis tools, with findings limited to the tested builds, devices, and configurations.

Concern	Treatment
Reliability	Repeat performance operations and preserve attack scripts, logs, failed attempts, and extracted artifacts.
Reproducibility	Record SDK and engine versions, dependencies, OS build, hardware, build flags, workloads, configuration, and script revisions.

3.17 Ethical Considerations

The experiments will use an application and devices controlled by the researcher, with synthetic datasets and test credentials. The planned procedures do not require access to third-party applications or real user accounts.

Logs and screenshots will be checked before inclusion in the report to remove unrelated personal information. The released evidence will identify the test configuration and preserve the synthetic values needed to reproduce each recovery or modification result.

3.18 Expected Results

The project is expected to produce a plugin, an envelope specification, integration examples, and a reproducible comparison dataset. The experiments should show which stored artifacts remain readable, which key paths can be intercepted, and which modifications are rejected.

The outcome may vary by access path. The scheme may protect an extracted file while a hook captures its contents after decryption. That result will be reported together with the root-key behavior, recovered material, and interception point. Benchmarks will provide the corresponding storage and resource costs.

3.19 Proposed Timeline

Weeks	Main tasks
1–3	Finalize assets, access assumptions, scope, variables, measurements, and baseline configurations.
4–6	Complete the related-work review and check tool support for the selected Flutter builds.
7–10	Specify the key hierarchy, envelope, authenticated context, migration rules, and public API.
11–15	Implement the plugin and reference application; verify storage operations and failure handling.
16–19	Run extraction, instrumentation, modification, rollback, and repackaging experiments; verify any resulting fixes.
20–21	Benchmark supported operations, record integration effort, and analyze the comparison data.

Weeks	Main tasks
22–24	Write the results and discussion, complete the thesis, prepare the defense, and package the implementation and evidence.

PROPOSAL READINESS CHECKLIST

The checklist records the items specified in the proposal.

Status	Item	Location or evidence
Defined	Research problem	Chapter 1 identifies local persistence, key handling, and the attack conditions to investigate.
Defined	Research questions	The questions are linked to recovery outcomes, overhead, integration records, or control-verification evidence.
Specified	Artifact	The planned outputs are a Flutter plugin, reference application, envelope specification, and integration examples.
Specified	Baselines	Five configurations cover plaintext, obfuscation, secure key-value storage, SQLCipher, and field encryption.
Planned	Experimental procedure	Chapter 3 defines operations, attack scenarios, configuration records, measurements, and evidence logging.
Organized	Literature review	Chapter 2 groups Flutter analysis, encrypted storage, platform keys, runtime interception, and technical references.
Planned	Quantitative analysis	Warm-up runs, repeated measurements, descriptive statistics, attack denominators, and comparison conditions are specified.
Stated	Scope and limitations	Android is the primary test platform; runtime exposure and device-dependent behavior are included in the limits.

CONCLUSION

This project will implement a Flutter storage scheme and test its key, payload, and metadata handling under defined reverse-engineering conditions. The comparison will examine extracted artifacts, cryptographic execution paths, substituted records, older valid records, and repackaged builds.

The thesis will report each recovery or acceptance outcome with its access conditions and supporting evidence. Latency, resource use, package size, and integration records will accompany the security results for the supported workloads.

REFERENCES

[1] Flutter Documentation, “Obfuscate Dart Code,” Flutter Docs. Available: https://docs.flutter.dev/deployment/obfuscate

[2] OWASP, “Mobile Application Security Testing Guide (MASTG),” OWASP Mobile Application Security Project. Available: https://mas.owasp.org/MASTG/

[3] OWASP MASTG, “Reverse Engineering Flutter Applications.” Available: https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0112/

[4] OWASP, “MASVS-STORAGE: Secure Storage,” OWASP Mobile Application Security Project. Available: https://mas.owasp.org/MASVS/controls/MASVS-STORAGE/

[5] OWASP, “MASVS-CRYPTO: Cryptography,” OWASP Mobile Application Security Project. Available: https://mas.owasp.org/MASVS/controls/MASVS-CRYPTO/

[6] OWASP, “MASVS-RESILIENCE: Resilience Against Reverse Engineering and Tampering,” OWASP Mobile Application Security Project. Available: https://mas.owasp.org/MASVS/controls/MASVS-RESILIENCE/

[7] B. Batteux, “The Current State and Future of Reversing Flutter Apps,” Guardsquare, 2022.

[8] PNF Software, “Dart AOT Snapshot Helper Plugin to Better Analyze Flutter-based Apps,” 2022, updated 2023.

[9] W. Wangwarunyoo, “B(l)utter: Reversing Flutter Applications by Using Dart Runtime,” HITB Security Conference, 2023.

[10] M. W. Wijaya and A. Kurniawan, “Exploring Dynamic Security Analysis of Android Apps Across Flutter Versions,” 2025 International Conference on Innovations in Intelligent Systems: Advancements in Computing, Communication, and Cybersecurity (ISAC3), 2025. doi: 10.1109/ISAC364032.2025.11156474. Research artifacts: https://github.com/PlasmaRing/DSA-Flutter-Android-Research and https://doi.org/10.5281/zenodo.15816265.

[11] S. Llewellyn, “Exploring Vulnerabilities in Flutter Mobile Apps Through Reverse Engineering,” COMMSEC, 2024.

[12] P. Rambhia, P. Shinde, and K. Bamane, “Securing Flutter Applications: A Comprehensive Study,” in 2023 7th International Conference on Computing, Communication, Control and Automation (ICCUBEA), 2023, doi: 10.1109/ICCUBEA58933.2023.10392001.

[13] C. V. T. Thao, T. D. Thi, C. Le, et al., “Security Techniques in Cross-Platform Software Development Based on the Flutter Framework,” 2025. doi: 10.1109/EEE-AM66675.2025.11473565.

[14] pub.dev, “flutter_secure_storage Package Documentation.” Available: https://pub.dev/packages/flutter_secure_storage

[15] Android Developers, “Android Keystore System.” Available: https://developer.android.com/privacy-and-security/keystore

[16] Apple Developer Documentation, “Keychain Services.” Available: https://developer.apple.com/documentation/security/keychain-services

[17] J. Blessing, R. J. Anderson, and A. R. Beresford, “KeyDroid: A Large-Scale Analysis of Secure Key Storage in Android Apps,” arXiv:2507.07927, 2025, doi: 10.48550/arXiv.2507.07927. Available: https://arxiv.org/abs/2507.07927

[18] J. H. Park, S. M. Yoo, I. S. Kim, and D. H. Lee, “Security Architecture for a Secure Database on Android,” IEEE Access, vol. 6, pp. 11482–11501, 2018, doi: 10.1109/ACCESS.2018.2799384.

[19] F. Akowuah, A. Ahlawat, and W. Du, “Protecting Sensitive Data in Android SQLite Databases Using TrustZone,” 2018.

[20] Zetetic, “SQLCipher Documentation.” Available: https://www.zetetic.net/sqlcipher/

[21] C. Anglano, M. Canonico, A. Cepollina, D. Freggiaro, A. Gallo, and M. Guazzone, “Enabling the Forensic Study of Application-Level Encrypted Data in Android via a Frida-Based Decryption Framework,” ARES 2023. doi: 10.1145/3600160.3605029.

[22] NIST SP 800-38D, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC.” Available: https://csrc.nist.gov/publications/detail/sp/800-38d/final

[23] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, 2010. Available: https://www.rfc-editor.org/rfc/rfc5869

[24] P. Wang, Q. Bao, L. Wang, S. Wang, Z. Chen, T. Wei, and D. Wu, “Software Protection on the Go: A Large-Scale Empirical Study on Mobile App Obfuscation,” in Proceedings of the 40th International Conference on Software Engineering (ICSE), pp. 26–36, 2018, doi: 10.1145/3180155.3180169.

[25] O. Mykhaylova, T. Fedynyshyn, and A. Platonenko, “Hardcoded Credentials in Android Apps: Service Exposure and Category-based Vulnerability Analysis,” Cybersecurity Providing in Information and Telecommunication Systems II, CEUR Workshop Proceedings, vol. 3826, pp. 206–211, 2024. Available: https://ceur-ws.org/Vol-3826/short8.pdf

[26] D. Schmidt, S. Schrittwieser, and E. R. Weippl, “Leaky Apps: Large-scale Analysis of Secrets Distributed in Android and iOS Apps,” in Proceedings of the 32nd ACM SIGSAC Conference on Computer and Communications Security (CCS), 2025, doi: 10.1145/3719027.3765033.